

자료구조와 알고리즘

Data Structures and Algorithms

2주차 (1) · 자료구조

2026-2 학기 · 김지언

목차

01 자료구조 기초

- 자료구조란?
- 자료구조를 배우는 이유
- 자료구조의 분류
- 자료구조와 알고리즘

02 자료구조와 메모리

- RAM과 데이터 저장 구조
- 비트 · 바이트 단위
- 2진법 · 16진법 표현
- 메모리 주소와 자료형 크기

03 선형 구조

- 리스트와 배열
- 연결 리스트
- 스택 · 큐

04 비선형 구조

- 해시테이블과 충돌 처리
- 트리와 이진 트리 순회
- 이진 탐색 트리 (BST)
- 그래프와 표현 방법
- 그래프 탐색 (BFS · DFS)

자료구조란?

- 자료를 효율적으로 사용하기 위해서 자료의 특성에 따라서 분류하여 구성하고 저장 및 처리하는 모든 작업

배열



큐

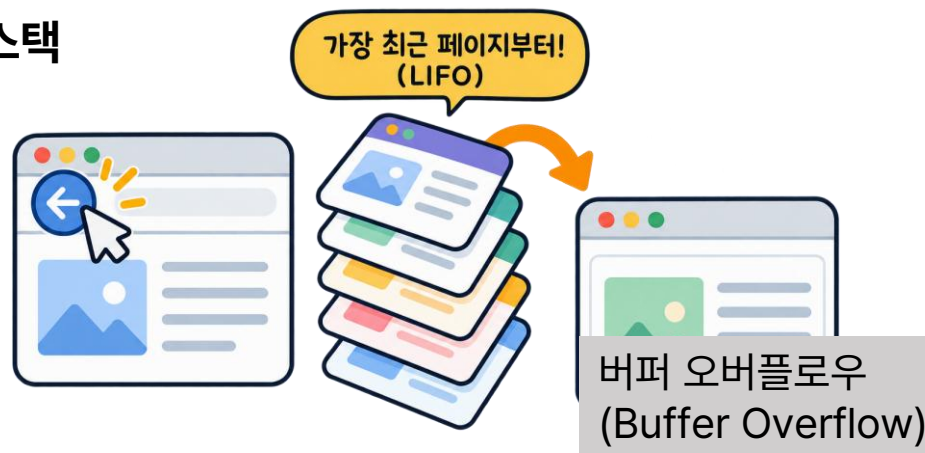


알고리즘 복잡도 공격,
해시 충돌, 무차별 대입..

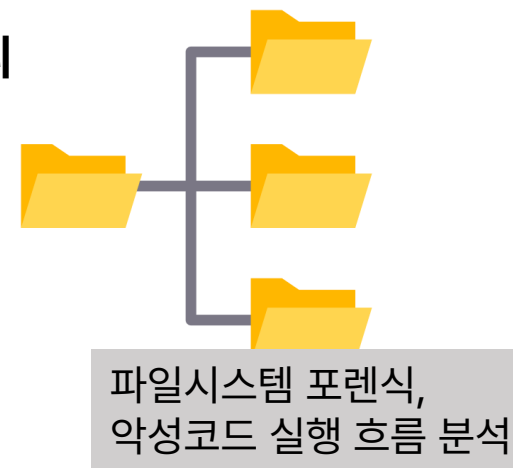
그래프



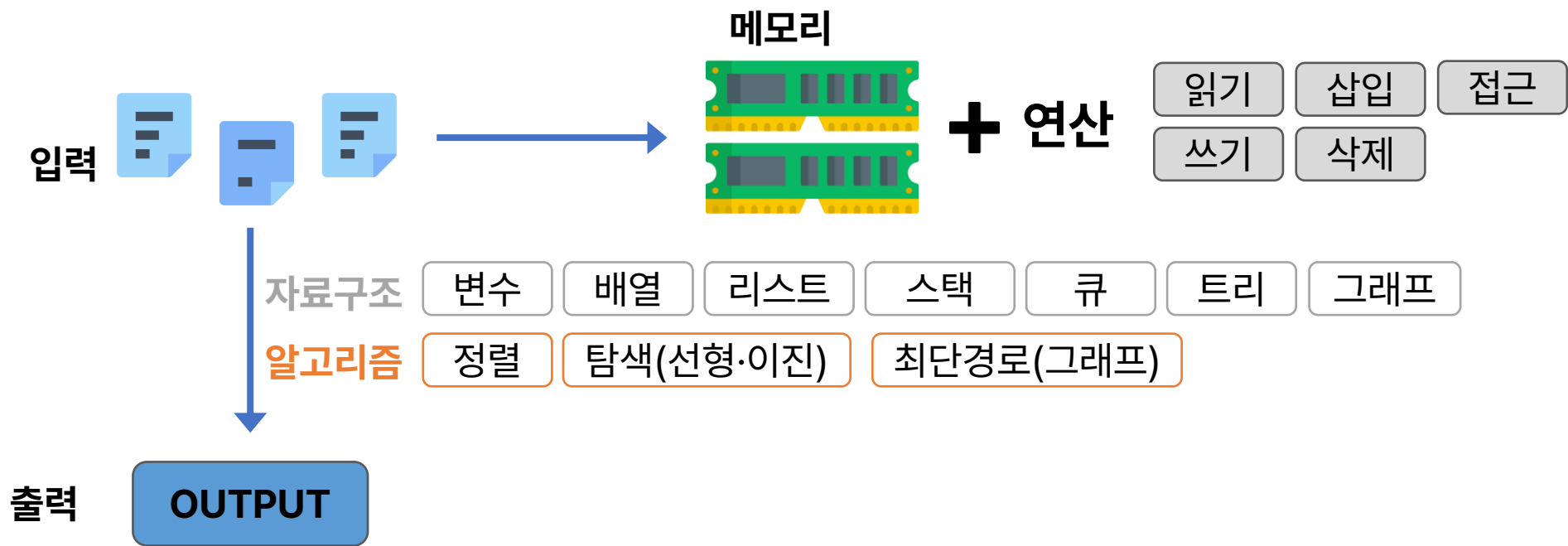
스택



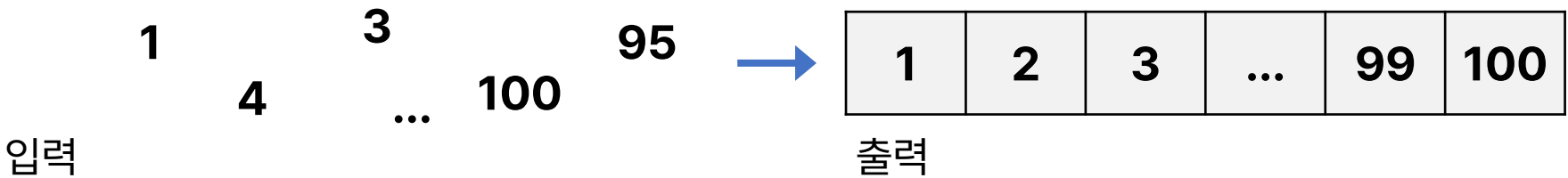
트리



자료구조란?



■ 예시



자료구조를 배우는 이유

- 컴퓨터와 인간의 사고 처리는 다름



자료구조의 분류

자료의 형태에 따른 분류

■ 단순 구조

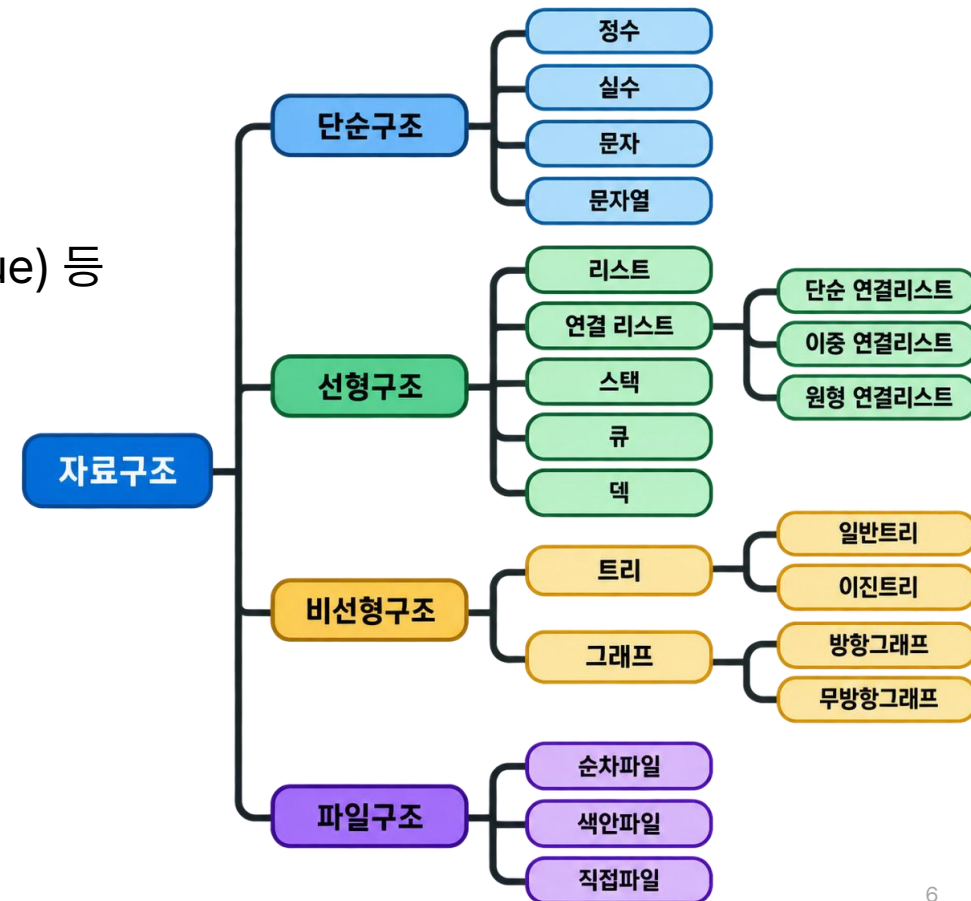
- 정수(integer), 실수(floating), 문자(char), 문자열(string) 등의 기본 자료형

■ 선형 구조

- 자료 간 앞뒤 관계가 1:1인 선형 관계
- 리스트(list), 연결리스트(linked list), 스택(stack), 큐(queue), 덱(deque) 등

■ 비선형 구조

- 자료 간 앞뒤 관계가 1:N 또는 N:N의 관계
- 트리(tree), 그래프(graph), 해시테이블(hash table) 등



자료구조와 알고리즘

알고리즘이란?

- 어떤 문제를 해결하기 위한 절차

- 데이터를 담는 방법 = 자료구조 / 담은 데이터로 문제를 푸는 방법 = 알고리즘

- 알고리즘 조건

1. 입력 (Input) – 0개 이상의 입력이 존재해야 함
2. 출력 (Output) – 최소 1개 이상의 결과를 출력해야 함
3. 명확성 (Definiteness) – 각 명령어는 모호하지 않고 명확해야 함
4. 유한성 (Finiteness) – 유한한 단계 안에 반드시 종료되어야 함 (무한 루프 – 알고리즘 X)
5. 효과성/실행 가능성 (Effectiveness) – 각 단계가 실제로 실행 가능해야 함

- 알고리즘 선택

- 주어진 문제를 해결하기 위해 가능한 빨리, 적은 자원으로 문제에 적용

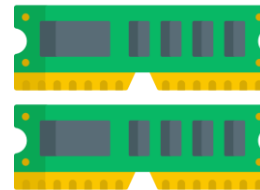
자료구조와 메모리

RAM (Random Access Memory)

```
int a = 3  
int b = 4  
print ("hello world")  
print(a + b)
```



SSD/HDD



RAM

디지털 데이터의 저장과 표현

- | Offset(h) | 00 | 01 | 02 | 03 | 04 | 05 | 06 | 07 | 08 | 09 | 0A | 0B | 0C | 0D | 0E | 0F | Decoded text |
|-----------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|--------------------|
| 00000000 | 89 | 50 | 4E | 47 | 0D | 03 | 1A | 04 | 00 | 00 | 00 | 0D | 49 | 48 | 44 | 52 | PNP.....IHDR |
| 00000010 | 01 | 01 | 01 | 11 | 11 | 11 | 01 | 01 | 11 | 01 | 01 | 10 | 00 | 01 | | | ...E.....Z. |
| 00000020 | BC | 00 | 00 | 20 | 00 | 49 | 44 | 41 | 54 | 78 | 01 | EC | BD | 51 | 92 | 24 | 4... .IDATx.i4Q/\$ |
| 00000030 | 01 | 11 | 11 | 01 | 01 | 01 | 01 | 11 | 11 | 01 | 11 | 10 | 01 | 11 | | | Ze.U.U"Hyil,6. |
| 00000040 | 8B | 16 | 5F | 31 | 92 | 21 | BE | 95 | 37 | 50 | 00 | 49 | 35 | A3 | 9A | D1 | <x_l,n%*7..I5EŠN |
| 00000050 | 11 | 10 | 10 | 10 | 10 | 10 | 10 | 11 | 11 | 11 | 11 | | | | | | í.Š"...@!30+0+u |
| 00000060 | D7 | 7F | FE | F3 | 5F | 3F | 3C | 30 | E0 | 0C | 70 | 06 | 38 | 03 | 9C | 01 | x.póY?<0A.p.8.α |
| 00000070 | 01 | 01 | 01 | 11 | 11 | 11 | 01 | 01 | 11 | 01 | 01 | 10 | 00 | 01 | | | í.ge3A.à.p.8.α.í |
| 00000080 | 01 | 01 | 01 | 11 | 11 | 11 | 01 | 01 | 11 | 01 | 01 | 10 | 00 | 01 | | | .ge3A.Đ.α.δ.pYy- |
| 00000090 | 11 | 10 | 10 | 10 | 10 | 10 | 10 | 10 | 00 | 00 | 01 | 10 | 10 | 11 | | | ..8.α.í.ge3A.à.p |
| 000000A0 | 00 | 10 | 00 | 01 | 10 | 11 | 11 | 10 | 00 | 00 | 01 | 10 | 10 | 11 | | | .8.α.í.ge3A.à.p |
| 000000B0 | 00 | 10 | 00 | 01 | 10 | 11 | 11 | 10 | 00 | 00 | 01 | 10 | 10 | 11 | | | t.~ýyáí÷i8/ei(M |
| 000000C0 | 00 | 10 | 00 | 01 | 10 | 11 | 11 | 10 | 00 | 00 | 01 | 10 | 10 | 11 | | | >÷<ý9{.}auæ±00E |
| 000000D0 | 00 | 10 | 00 | 01 | 10 | 11 | 11 | 10 | 00 | 00 | 01 | 10 | 10 | 11 | | | B...E... ..@..YC |
| 000000E0 | 00 | 10 | 00 | 01 | 10 | 11 | 11 | 10 | 00 | 00 | 01 | 10 | 10 | 11 | | | €.ΔC8'.AC6<y.ú0E |
| 000000F0 | 01 | 01 | 01 | 11 | 11 | 11 | 01 | 01 | 11 | 01 | 01 | 10 | 00 | 01 | | | !4..gl.J.A...'.e. |
| 00000100 | 01 | 01 | 01 | 11 | 11 | 11 | 01 | 01 | 11 | 01 | 01 | 10 | 00 | 01 | | | ...@...pa'..āZ. |
| 00000110 | 7C | 2C | EE | A3 | 78 | 5F | 0F | F8 | E7 | 3E | B6 | 74 | FE | 22 | C6 | 19 | ,í x_δ<gtpbA |

2진법 (binary)



자료구조와 메모리

디지털 데이터의 저장과 표현

- 물리적 단위: 실제 저장 장치에서의 저장 단위
 - Bit, Byte, KB(Kilobyte), MB(Megabyte), TB(Terabyte) 등
- 논리적 단위: 정보를 저장 및 처리하는데 사용되는 최소 단위
 - 레코드(Record), 블록(Block), 파일(File), 데이터베이스(Database) 등

2진수	10진수	16진수
0000	0	0
0001	1	1
0010	2	2
0011	3	3
0100	4	4
0101	5	5
0110	6	6
0111	7	7
1000	8	8
1001	9	9
1010	10	a
1011	11	b
1100	12	c
1101	13	d
1110	14	e
1111	15	f

10진법

220

2진법

0b11011100
 └──┬──┘ └──┬──┘
 13 12

16진법

0xDC

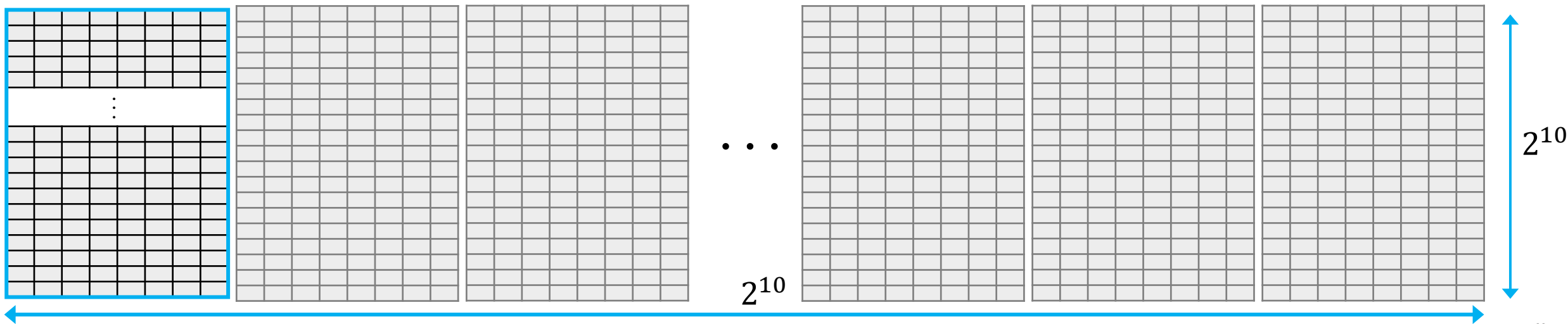
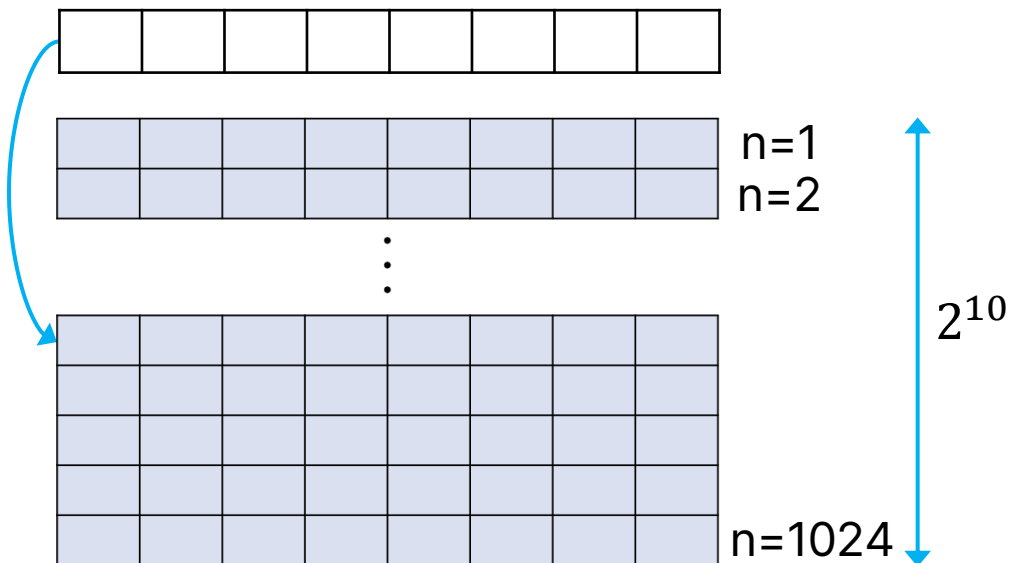
자료구조와 메모리

디지털 데이터의 저장과 표현

- 1bit 가 8개 모이면 = 8 bit = 1 byte

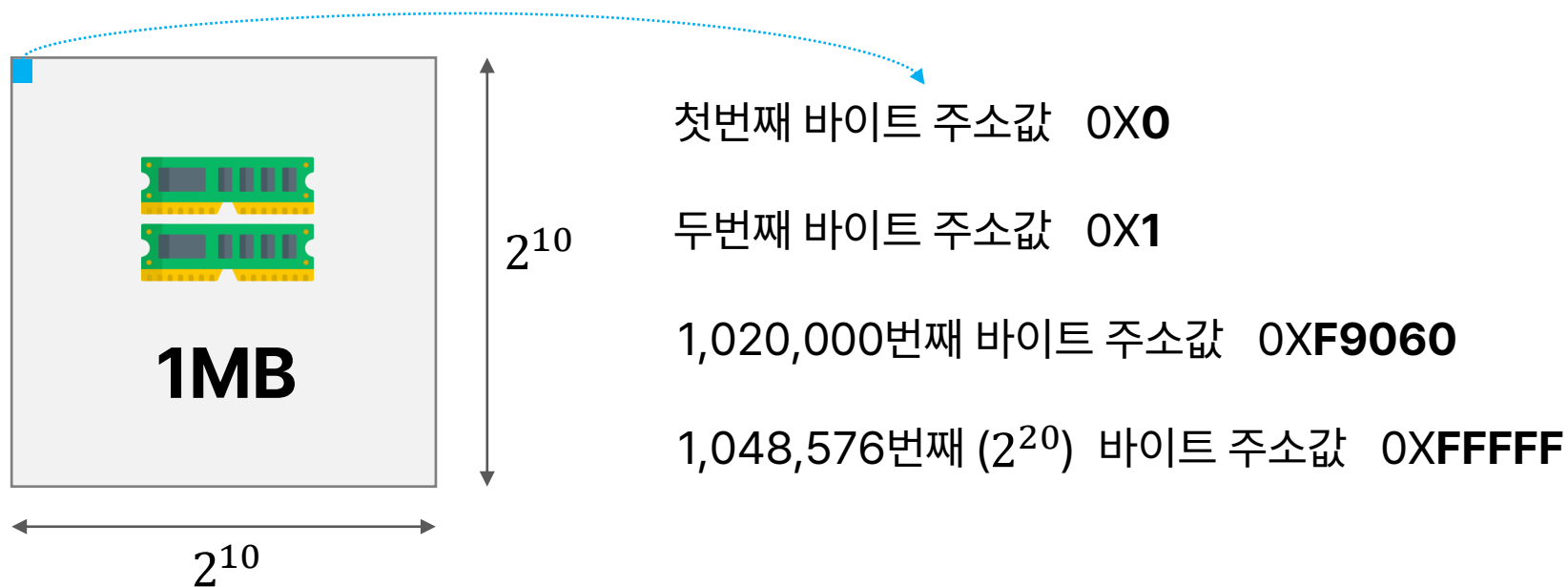
- 1 byte 가 1024개 모이면 = 1 KB

- 1 KB 가 1024개 모이면 = 1 MB



자료구조와 메모리

디지털 데이터의 저장과 표현



자료구조와 메모리

디지털 데이터의 저장과 표현

- 정수를 표현할 때는 4바이트를 차지함

```
int price = 1000;
```

2진수 변환

0x3AE10

0	0	0	0	0	0	0	0
---	---	---	---	---	---	---	---

0x3AE11

0	0	0	0	0	0	0	0
---	---	---	---	---	---	---	---

0x3AE12

0	0	0	0	0	0	1	1
---	---	---	---	---	---	---	---

0x3AE13

1	1	1	0	1	0	0	0
---	---	---	---	---	---	---	---

16진수 변환

$$\begin{array}{cccc}
 3 & \times & 16^2 & + & E(14) & \times & 16^1 & + & 8 & \times & 16^0 & = & 3E8_{16} \\
 768 & & & & 224 & & & & 8 & & & &
 \end{array}$$

10진수

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F

16진수

- 문자를 표현할 때는 1바이트 차지함

- 문자 A 를 변수에 저장할 때 → 메모리에 숫자 65가 저장

0x3AE14

0	1	0	0	0	0	0	1
---	---	---	---	---	---	---	---

Dec	Hex	Chr
64	40	@
65	41	A
66	42	B
67	43	C
68	44	D

Dec	Hex	Chr
69	45	E
70	46	F
71	47	G
72	48	H
73	49	I

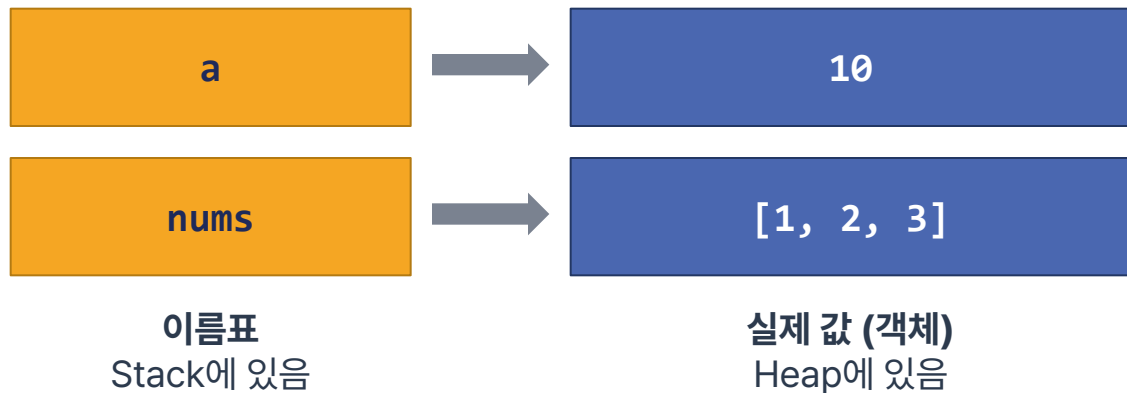
자료구조와 메모리

변수는 어디에 저장될까?

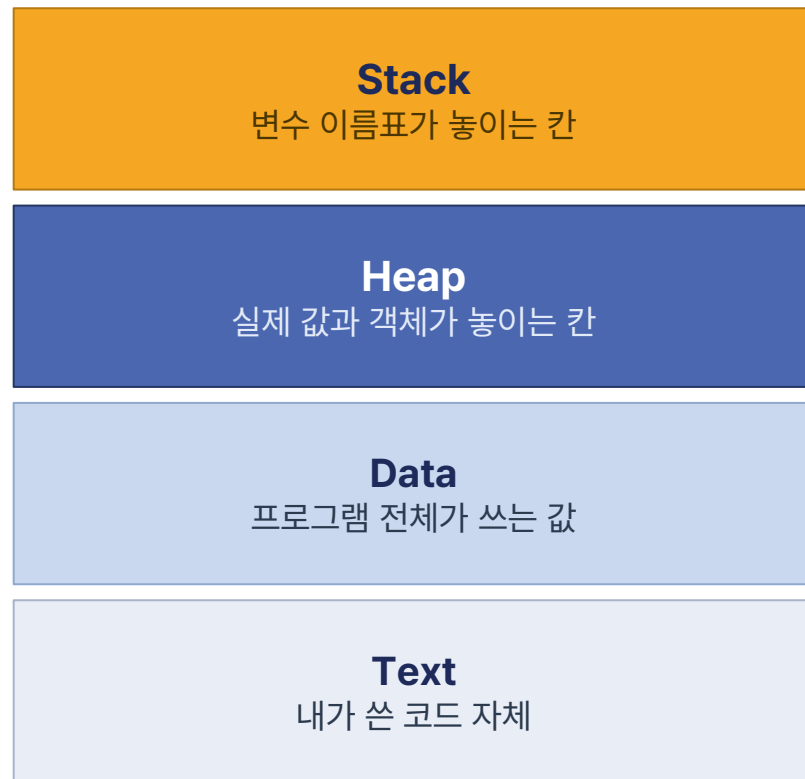
예제)

```
a = 10  
nums = [1, 2, 3]
```

이름과 실제 값은 따로 저장됩니다



메모리 영역



파이썬에서는 숫자·리스트·객체 모두 Heap에 저장됨
Stack에는 "어디에 있는지"

자료구조와 메모리

Heap은 왜 필요할까?

질문: 학생이 몇 명 올지 모르는데, 이름을 어떻게 저장할까요?

파이썬 리스트는 그냥 늘어납니다

```
names = []  
names.append("민수")  
names.append("지영")
```

→ 크기를 미리 정하지 않았습니다

append 할 때마다 파이썬이 Heap에서 필요한 만큼 공간을 더 가져옵니다.
그래서 실행 중에 늘어날 수 있습니다

이게 가능한 이유

Stack은 크기가 정해진 칸이라 늘릴 수 없지만, Heap은 실행 중에 필요한 만큼 가져올 수 있습니다

다 쓴 값은 어떻게 되나요?

```
names = None
```

이름표가 사라지면 파이썬이 알아서 Heap을 정리해 줍니다 (가비지 컬렉션).
우리가 직접 반납하지 않습니다

Stack = 크기가 정해진 사물함 / Heap = 필요한 만큼 빌려 쓰는 창고
(함수 끝나면 사라짐)

자료구조와 메모리

Heap의 공간 할당



실제 데이터 (값)에 대한 이름표 수가 0이 되면 파이썬이 Heap 공간을 반납함

자료구조와 메모리

메모리의 Heap과 자료구조의 Heap은 이름만 같고 다름

Heap — 이름만 같고, 완전히 다름

메모리의 힙 (Heap)

운영체제가 주는 "빈 공간"

리스트와 객체가 실제로 놓이는 그 영역

정렬 규칙 없음 (참고)

자료구조의 힙 (Heap)

트리 모양의 자료구조

가장 크거나 작은 값을 빨리 꺼내는 용도

우선순위 큐를 만들 때 활용 (뒤에서 배움)

Stack — 이름도 같고, 개념도 같음

둘 다 "마지막에 넣은 것을 먼저 꺼내는" 구조 (LIFO)

메모리의 Stack 영역도 함수가 호출된 순서의 역순으로 정리되기 때문에 같은 이름을 씁니다

선형 구조

선형 구조란?

- 데이터가 한 줄로 늘어서 앞뒤 관계가 정해짐
- 순서대로 메모리에 담으면 → 순차 리스트

요일 리스트	
1	월
2	화
3	수
4	목
5	금

버스 정류장	
1	강남역
2	역삼역
3	선릉역
4	삼성역
5	잠실역

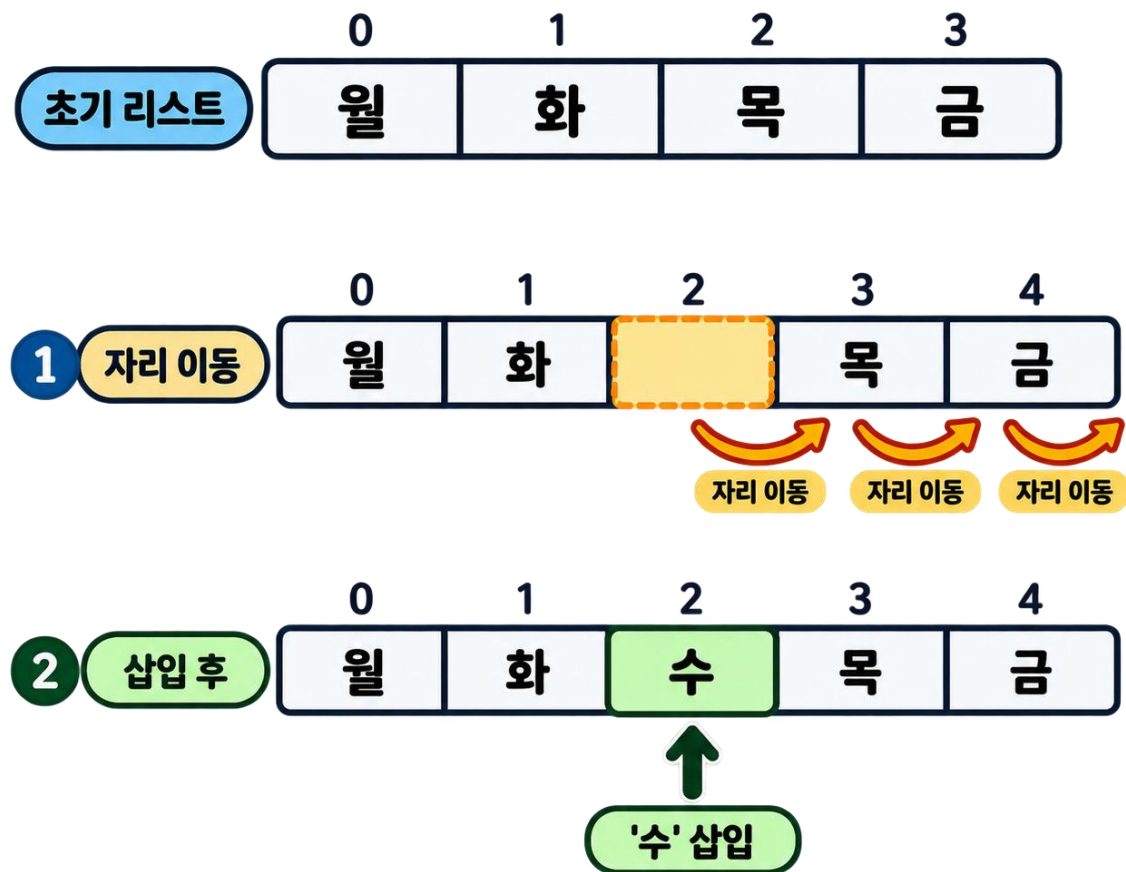
오늘 할 일	
1	운동하기
2	자료구조 수업
3	과제 제출
4	동아리 모임
5	방청소

선형 구조

순차 리스트에서 삽입이 일어나는 과정

- 원소(데이터) 삽입 방법

- 원소를 삽입할 빈 자리 만들기
- 빈 자리에 원소 삽입하기



선형 구조

순차 리스트는 메모리에 어떻게 올라갈까?

- 메모리에 연속적으로 저장
- 접근 속도 빠름
- 삽입/삭제 느림 (원소를 밀거나 당겨야 함)
- 크기 미리 고정해야 함

```
tabs = [ {"url": "https://google.com", "title": "구글"},  
         {"url": "https://naver.com", "title": "네이버"},  
         {"url": "https://github.com", "title": "깃허브"},  
         # ... 100개 탭  
       ]
```

```
int array[4] = {1, 2, 3, 4};
```

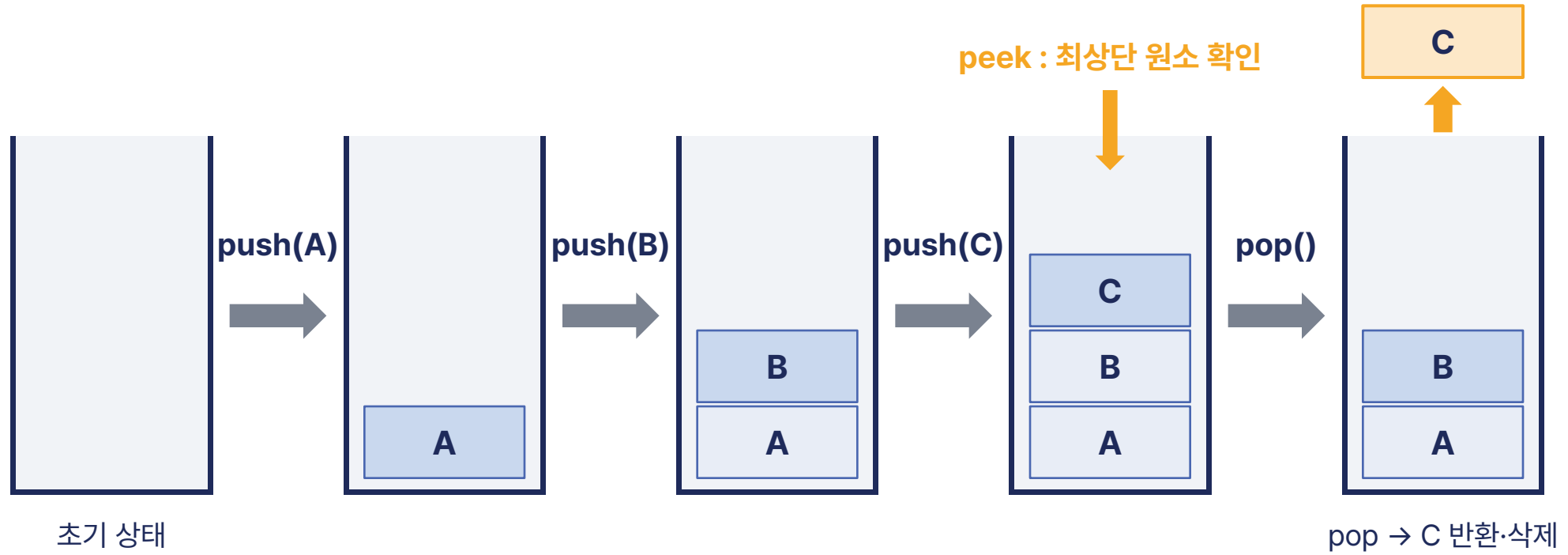
4byte each

0x3AE10	0	0	0	0	0	0	0	1
0x3AE11	0	0	0	0	0	0	0	
0x3AE12	0	0	0	0	0	0	0	
0x3AE13	0	0	0	0	0	0	1	
0x3AE14	0	0	0	0	0	0	0	2
0x3AE15	0	0	0	0	0	0	0	
0x3AE16	0	0	0	0	0	0	0	
0x3AE17	0	0	0	0	0	1	0	
0x3AE18	0	0	0	0	0	0	0	3
0x3AE19	0	0	0	0	0	0	0	
0x3AE1A	0	0	0	0	0	0	0	
0x3AE1B	0	0	0	0	0	1	1	
0x3AE1C	0	0	0	0	0	0	0	4
0x3AE1D	0	0	0	0	0	0	0	
0x3AE1E	0	0	0	0	0	0	0	
0x3AE1F	0	0	0	0	0	1	0	

선형 구조

스택 (stack)

- 후입선출(LIFO) 구조 : 가장 나중에 넣은 데이터를 가장 먼저 꺼냄
- 핵심 연산 : push(삽입) / pop(삭제) / peek(최상단 원소 확인)



선형 구조

스택 활용 : 함수 호출 스택 (call stack)

- 함수가 호출되면 스택에 쌓이고(push), 값을 반환하면 위에서부터 제거됨(pop)

예제 코드

```
function foo(b) {  
  var a = 5;  
  return a * b + 10;  
}  
  
function bar(x) {  
  var y = 3;  
  return foo(x * y);  
}  
  
console.log(bar(6));
```

출력 : 100

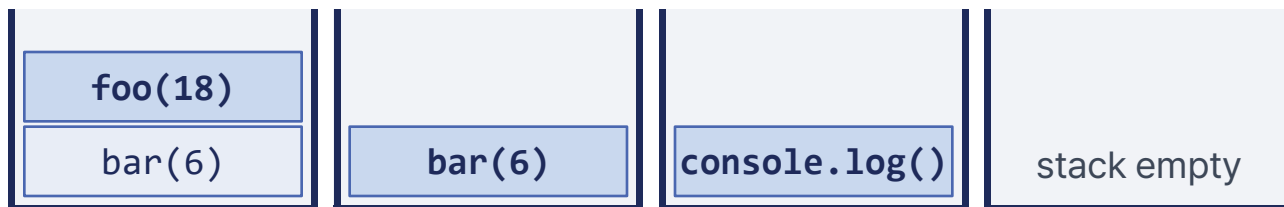
호출 스택의 변화

① push

② pop

③ push

④ empty



foo까지 쌓임

foo → 100 반환

bar 종료 → 출력 호출

출력 후 모두 pop

가장 나중에 호출된 함수가 가장 먼저 종료된다 → LIFO

선형 구조

스택의 사용 사례

- “가장 최근 것부터 되돌린다”는 동작은 대부분 스택으로 구현



브라우저 뒤로 가기

방문한 페이지를 차례로 push,
뒤로 가기를 누르면
가장 최근 페이지부터 pop



되돌리기 (Ctrl+Z)

편집 작업을 기록해 push,
되돌리기를 누르면
마지막 작업부터 취소



괄호 · 태그 검사

여는 괄호는 push,
닫는 괄호를 만나면 pop해
짝이 맞는지 확인



함수 호출 · 재귀

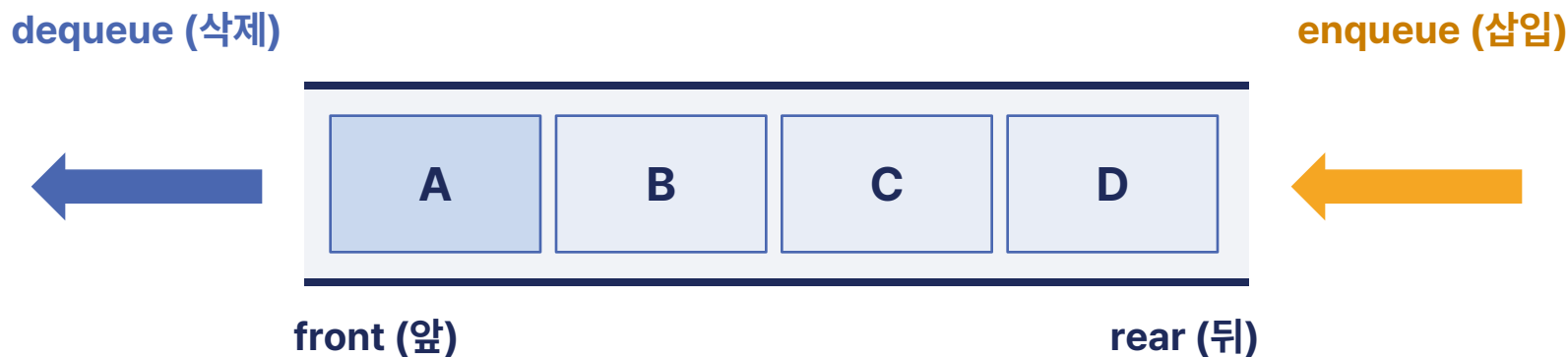
호출된 함수를 push,
반환되면 pop하는
호출 스택(call stack)

공통점 : 마지막에 들어온 것을 가장 먼저 되돌림 → LIFO

선형 구조

큐 (queue)

- 스택과 마찬가지로 삽입과 삭제의 위치가 제한된 유한 순서 리스트
- 뒤(rear)에서는 삽입만, 앞(front)에서는 삭제만 할 수 있는 구조
- 먼저 삽입한 원소가 먼저 삭제되는 선입선출(FIFO, First-In-First-Out) 구조

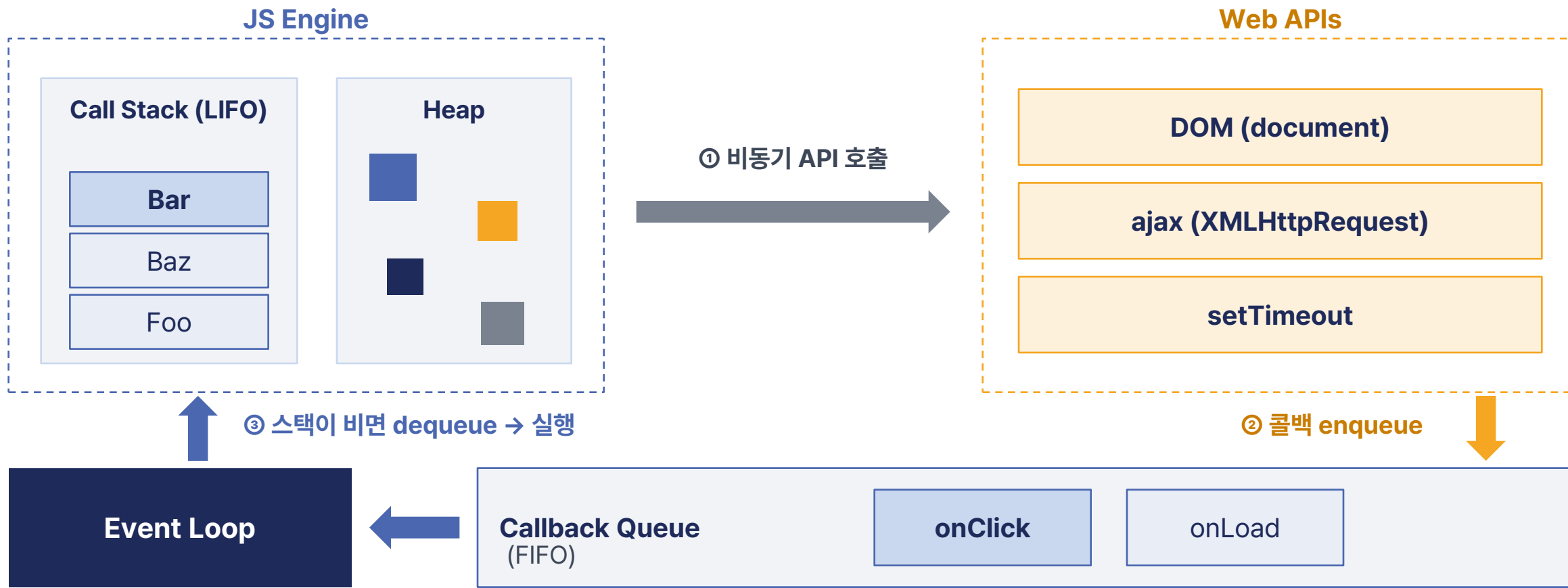


스택 : 후입선출(LIFO) ↔ 큐 : 선입선출(FIFO)

선형 구조

큐 실제 사용 예제 : 자바스크립트 이벤트 루프

- 비동기 작업의 콜백은 콜백 큐(FIFO)에 등록된 순서대로 콜 스택(LIFO)으로 옮겨져 실행

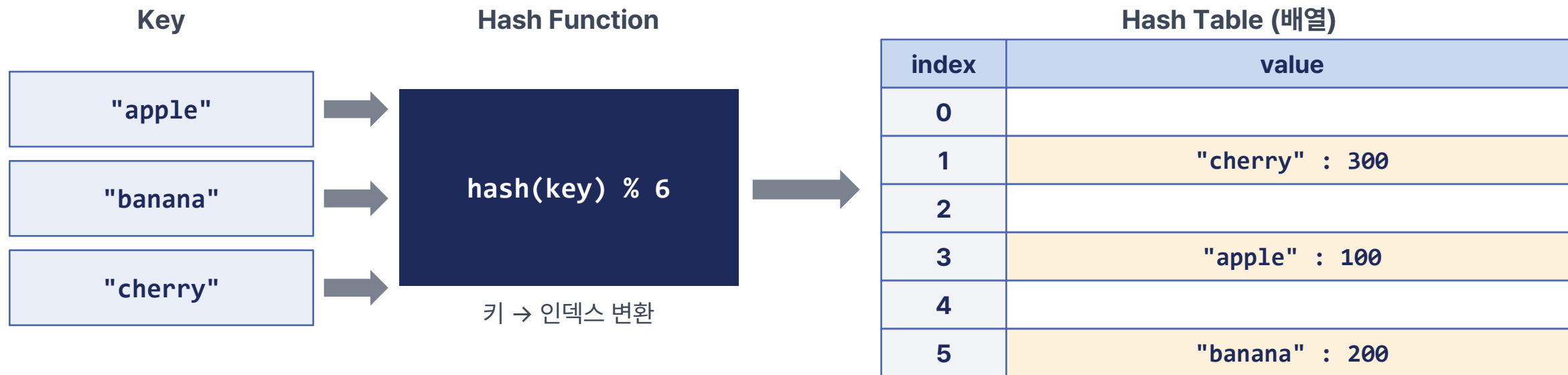


```
window.onload = function() { console.log("로딩 완료!"); };  
button.onclick = function() { console.log("클릭!"); };  
console.log("코드 끝");
```

비선형 구조

해시테이블 (hash table)

- 키(key)와 값(value)을 한 쌍으로 저장하는 자료구조 (map 구조)
- 해시 함수(hash function)가 키를 배열의 인덱스로 변환해 값을 저장 및 조회
- 탐색·삽입·삭제 평균 $O(1)$ (단, 저장 순서는 보장되지 않음)



서로 다른 키가 같은 인덱스로 변환되면 → 충돌(collision)

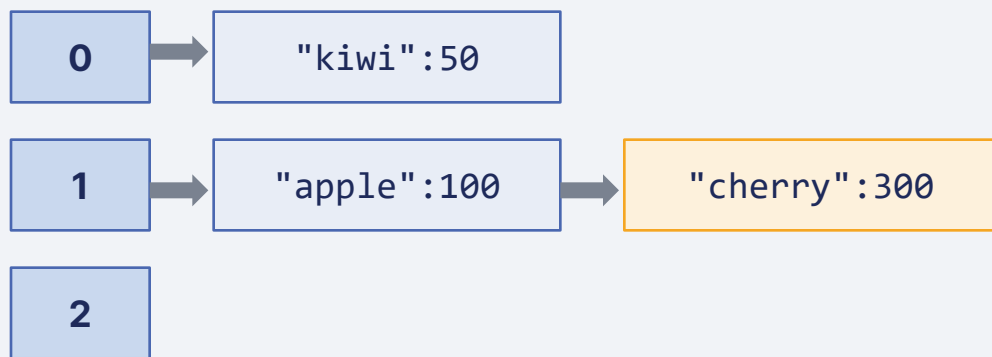
비선형 구조

충돌 (collision)과 해결 방법

- 충돌(collision) : 서로 다른 키가 같은 인덱스로 변환되는 상황

① 체이닝 (Separate Chaining)

같은 인덱스의 값들을 연결 리스트로 이어서 저장



충돌이 몰리면 리스트가 길어져 탐색이 $O(n)$ 에 가까워진다

② 개방 주소법 (Open Addressing)

빈 자리를 찾아 다음 인덱스에 저장 (선형 탐사)

0	
1	"apple" : 100
2	"cherry" : 300
3	

조회 : "apple"

① hash=1 확인
키 일치 → 100 반환

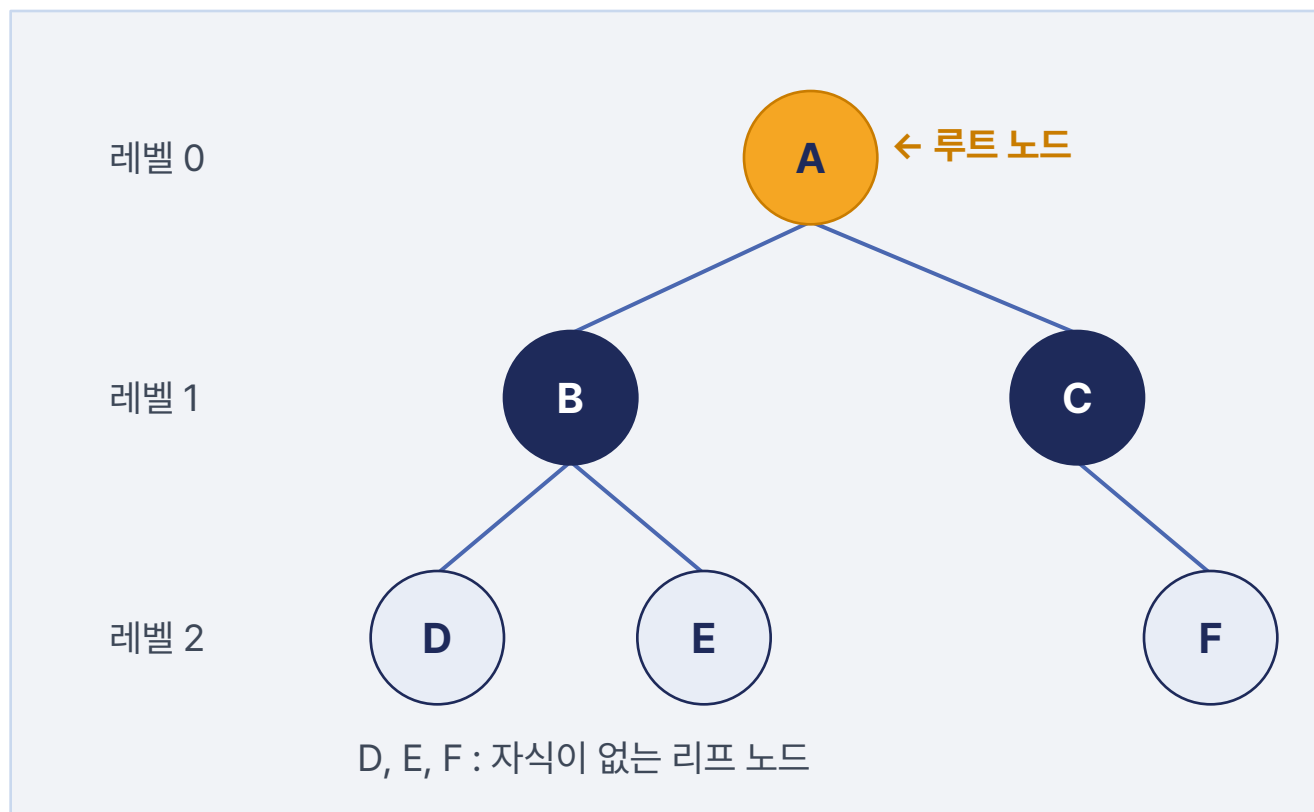
조회도 저장과 같은 규칙으로 탐사하며, 칸의 키를 비교해 확인한다

충돌을 줄이려면 : 좋은 해시 함수 + 적절한 적재율(load factor) 관리

비선형 구조

이진 트리 (binary tree)

- 데이터를 계층 구조(부모-자식)로 저장하는 자료구조



꼭 알아야 할 용어

루트(root)

가장 위에 있는 시작 노드 — A

부모 · 자식

바로 위 / 아래로 연결된 노드

리프(leaf)

자식이 없는 끝 노드 — D, E, F

레벨 · 높이

루트에서 내려온 단계의 수

서브트리

어떤 노드와 그 아래 전체

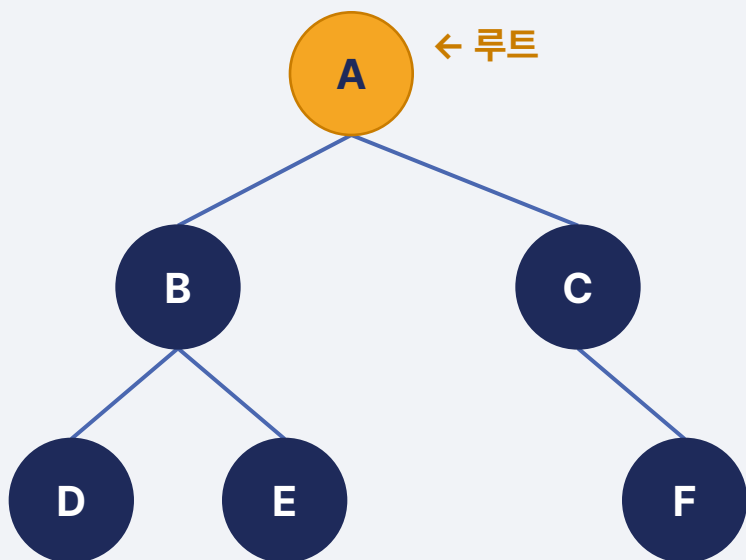
이진 트리 : 모든 노드가 왼쪽 · 오른쪽 자식을 최대 하나씩만 가짐 (자식 최대 2개)

비선형 구조

이진 트리 순회 (traversal)

- 모든 노드를 한 번씩 방문하는 방법

예시 트리



왼쪽을 오른쪽보다 먼저 방문하는 점은 세 방법 모두 같다

전위 순회 (preorder)

루트 → 왼쪽 → 오른쪽



주황 = 루트 A 방문 시점

중위 순회 (inorder)

왼쪽 → 루트 → 오른쪽



후위 순회 (postorder)

왼쪽 → 오른쪽 → 루트



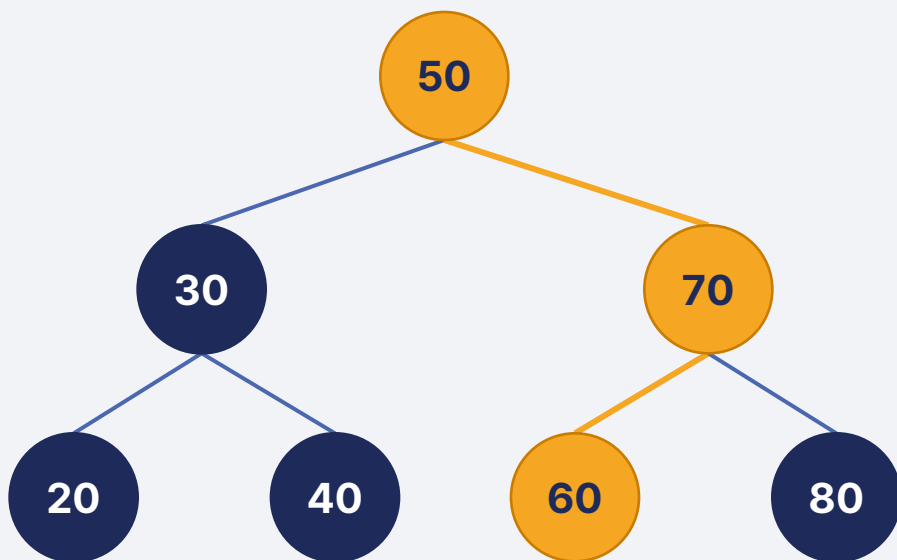
루트를 처음(전위) · 중간(중위) · 마지막(후위)에 방문함

비선형 구조

이진 탐색 트리 (BST, binary search tree)

- 왼쪽은 나보다 작은 값, 오른쪽은 나보다 큰 값 → 값을 빠르게 찾을 수 있음

이진 탐색 트리 예시



주황색 경로 = 60을 찾아가는 길 (비교 3번)

BST 의 규칙

- 왼쪽 자식 < 부모 노드
- 오른쪽 자식 > 부모 노드
- 모든 서브트리에서 이 규칙이 성립한다

60 을 찾는 과정

- 1 50과 비교 → 60이 더 크다 → 오른쪽으로
- 2 70과 비교 → 60이 더 작다 → 왼쪽으로
- 3 60 발견! 비교 3번으로 끝

비교할 때마다 후보가 절반으로 줄어든다 → 평균 $O(\log n)$

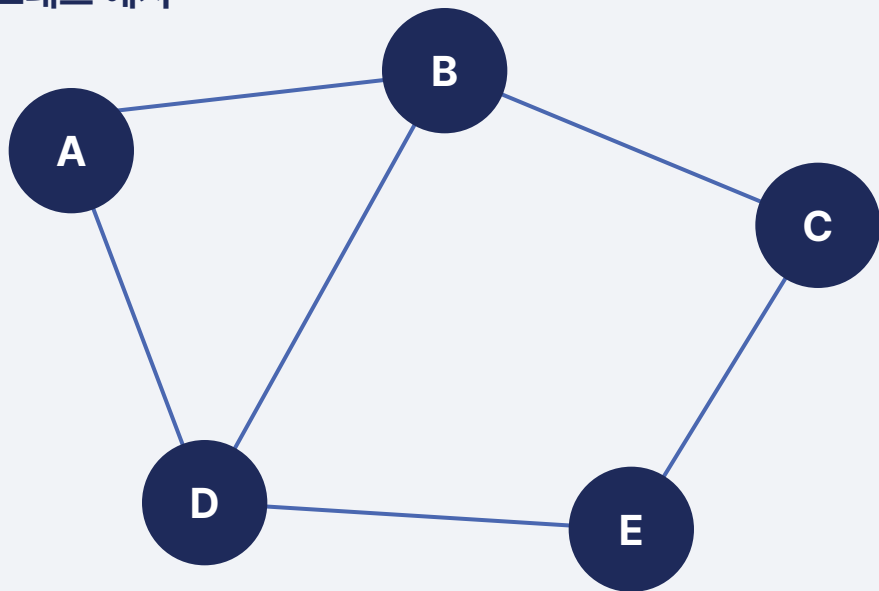
중위 순회하면 20 · 30 · 40 · 50 · 60 · 70 · 80 — 항상 정렬된 순서로 나옴

비선형 구조

그래프 (graph)

- 정점(vertex)과 간선(edge)으로 연결 관계를 표현하는 자료구조
 - 예시) 지하철 노선도, SNS 친구 관계

무방향 그래프 예시



점 = 정점(vertex), 선 = 간선(edge) : A와 B는 서로 연결되어 있다

기본 용어

정점 (vertex)	데이터를 담는 점 — A, B, C ...
간선 (edge)	두 정점을 잇는 선 = 관계
차수 (degree)	한 정점에 연결된 간선의 개수
경로 · 사이클	정점을 따라 이동한 길 / 제자리로 돌아오는 길

그래프의 종류

- 무방향 그래프 양쪽 모두 연결 — 친구 관계
- 방향 그래프 한쪽 방향만 — 팔로우, 웹 링크
- 가중치 그래프 간선에 거리 · 비용을 표시 — 지도 경로

트리도 사이클이 없는 그래프의 한 종류

비선형 구조

그래프를 저장하는 두 가지 방법

- 표로 담기 → 인접 행렬 / 목록으로 담기 → 인접 리스트

① 인접 행렬 (matrix)

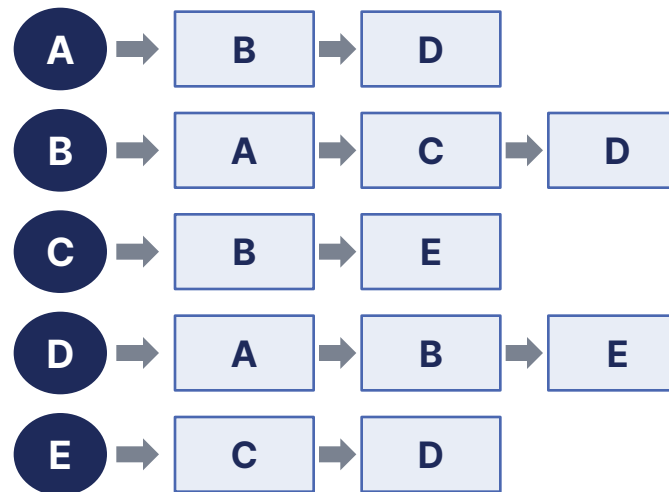
연결되어 있으면 1, 아니면 0

	A	B	C	D	E
A	0	1	0	1	0
B	1	0	1	1	0
C	0	1	0	0	1
D	1	1	0	0	1
E	0	0	1	1	0

메모리 정점² 만큼 필요 · 두 정점의 연결 여부를 바로 확인

② 인접 리스트 (list)

각 정점마다 이웃만 이어서 저장



메모리 정점+간선 만큼만 사용 · 이웃을 차례로 훑기에 좋다

간선이 촘촘하면 인접 행렬, 드문드문하면 인접 리스트가 유리함

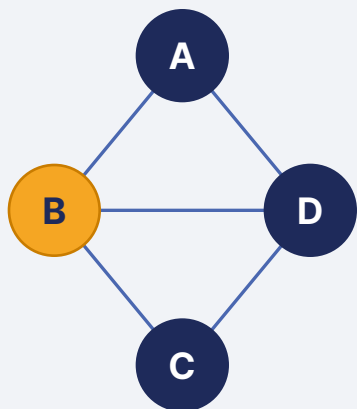
비선형 구조

인접 행렬 자세히 보기 : 구조와 차수

- 정점이 V 개면 $V \times V$ 크기의 2차원 배열을 만들고, i 에서 j 로 가는 간선이 있으면 $[i][j] = 1$ 로 표시한다

무방향 그래프

A-B, A-D, B-C, B-D, C-D



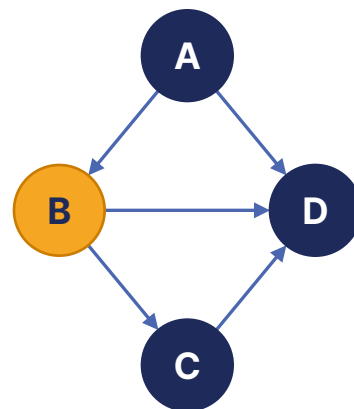
	A	B	C	D
A	0	1	0	1
B	1	0	1	1
C	0	1	0	1
D	1	1	1	0

B행 : $1+0+1+1 = 3 \rightarrow$ B의 차수

대각선 기준으로 대칭 : $arr[i][j] = arr[j][i]$, 대각선은 항상 0

방향 그래프

A→B, A→D, B→C, B→D, C→D



	A	B	C	D
A	0	1	0	1
B	0	0	1	1
C	0	0	0	1
D	0	0	0	0

B행 합 = 2 → 진출차수

B열 합 = 1 → 진입차수

대칭이 아니다

무방향은 행 합 = 차수 · 방향 그래프는 행 = 나가는 간선, 열 = 들어오는 간선

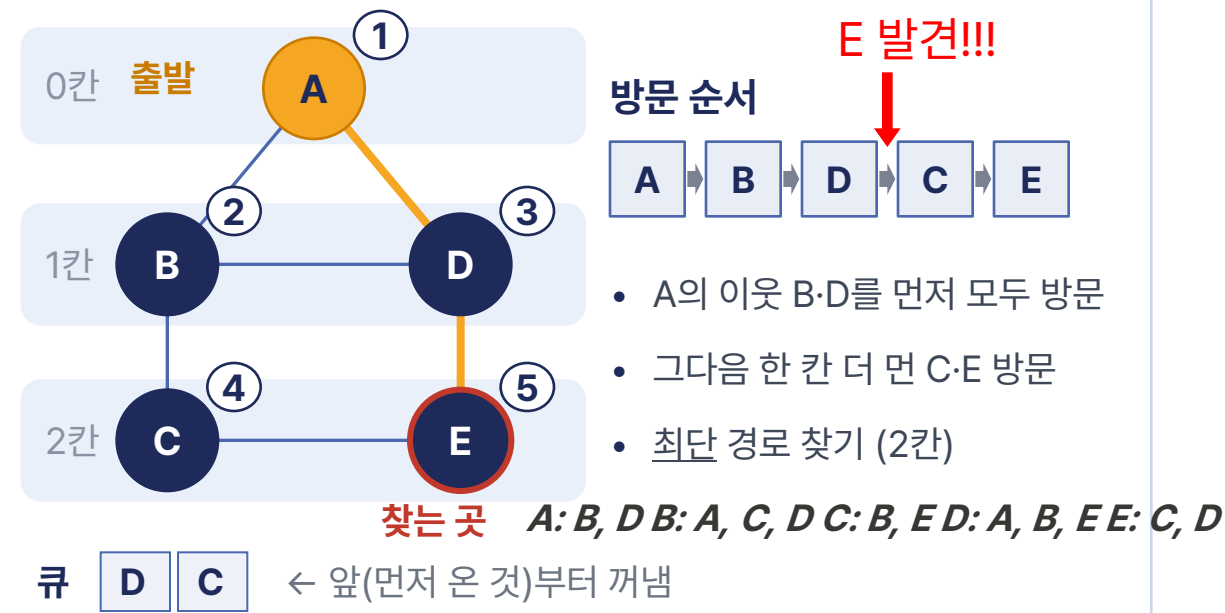
비선형 구조

그래프 탐색 : BFS 와 DFS

- 모든 정점을 빠짐없이 방문하는 방법
 - 큐를 쓰면 BFS, 스택(재귀)을 쓰면 DFS

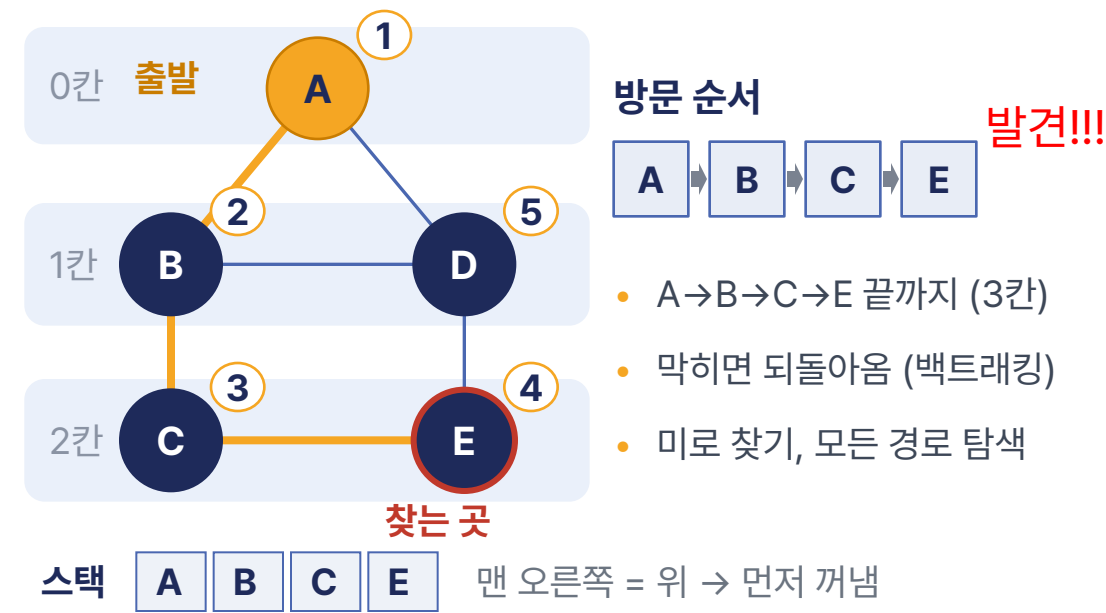
너비 우선 탐색 (BFS)

Breadth-First Search · 큐(FIFO) 사용 — 가까운 곳부터 넓게



깊이 우선 탐색 (DFS)

Depth-First Search · 스택(LIFO)·재귀 — 한 길로 끝까지



같은 그래프라도 무엇을 쓰느냐(큐 · 스택)에 따라 방문 순서가 달라짐

THANK YOU

Email | jieonkim@hallym.ac.kr
Tel. | 033-248-2353